THE UNIVERSITY OF SYDNEY

Tutorial Class	
Student Name	
SID	

CONFIDENTIAL QUIZ PAPER

This paper should not be removed from the quiz venue.

COMP5339 – Data Engineering
Tutorial Quiz – 1
Individual Quiz (**5%**)

Quiz Writing Time: **30 minutes**
Thursday, 16 April

Instructions to Students

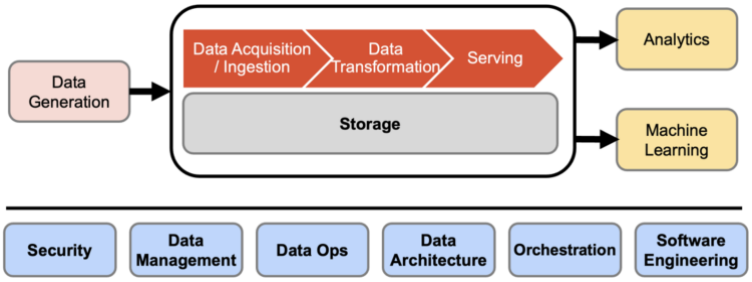
- Please ensure you correctly complete your **tutorial** and **Student ID** details.
- This is a **closed-book** test. **Electronic devices** (including phones, laptops, tablets, etc.) are **not permitted**. Please place all devices in your bag and keep your bag on the floor.
- This tutorial quiz consists of **3 multiple-choice questions** and three short descriptive questions. **All questions must be answered**.
- Write all answers in the spaces provided on this paper.
- Questions carry **different marks**. The mark value for each question is shown at the start of the question. The **total mark** for this quiz is **10**, which contributes to 5% of the total marks.
- For short-answer questions, please write **legibly**. Record your final answers in **ink**. **Do not** use a pencil or **red ink**.

Please submit the completed quiz paper to your tutor **before leaving the room**.

Part I – Multiple Choice Questions

Question 1: This question has three parts. Tick the boxes corresponding to each correct answer. Each question carries one mark.

A. In the "Data Engineering Lifecycle" diagram, why is Storage drawn as a long layer underneath ingestion → transformation → serving (instead of being a single step)?



- ☐ **Storage supports all stages (landing raw data, staging transformations, and serving curated data), and its choice affects performance, cost, and reliability across the pipeline**
- ☐ Storage is only used for archiving after analytics are complete
- ☐ Storage is mainly for backups and doesn't affect pipeline design
- ☐ Storage is only needed when working with streaming systems

B. When extracting data from a webpage, what is the most important first step?

- ☐ **Inspecting the webpage structure**
- ☐ Writing extraction code
- ☐ Storing data in a database
- ☐ Running the crawler

C. A data engineering team is building a pipeline to ingest data from multiple external APIs whose structures frequently change (new fields, missing fields, nested variations). Which design choice is most appropriate?

- ☐ Use a strictly normalised relational schema before ingestion
- ☐ **Use a schema-on-read approach with a document-oriented store**
- ☐ Use a schema-on-write approach with enforced constraints
- ☐ Use a columnar data warehouse with a fixed schema

Part II – Short Answer Questions

Provide answers in the provided boxes.

Question 1: Why is a graph database more suitable than a relational model for this use case? (**2 Marks**)

Graph databases efficiently represent relationships such as prerequisites and dependencies between concepts. They allow fast traversal without complex joins. This makes them ideal for ontology-driven systems and knowledge graphs where relationships are central.

Question 2: Data Acquisition, Cleaning, and Integration

Scenario 1: You work at an e-commerce company and ingest order events daily from multiple source systems. Sometimes an order arrives 2-3 days late, and occasionally the same order appears twice with identical fields.

Answer the following:
How would you design your ingestion + staging layer to ensure:

1. Non-identical loads (no duplicates), (**1 Mark**)
2. Correct historical completeness (late arrivals are included), and (**1 Mark**)
3. Identify a schema change? (**1 Mark**)

1. Store the data into the staging area and find any duplicates (i.e., the entire row is duplicated) before ingesting into a database or storage area

2. Look for the orders that were completed in the past and include them in their respective days/dates

3. Store the base schema in a storage area and compare it with the current schema at ingestion time

Scenario 2: The source team adds a new column (`discount`) to the orders feed and later changes the `postal_code` column type from numeric to string. Your pipeline starts failing.

Answer the following:
What controls and design choices would you put in place to handle schema evolution safely (both breaking and non-breaking changes) while keeping downstream analytical tables stable? (**2 Marks**)

Implement schema validation at ingestion and alert on breaking changes. Use schema evolution rules: allow additional fields, but fail on incompatible type changes. Store raw payloads to avoid data loss and map them to a stable schema in staging.